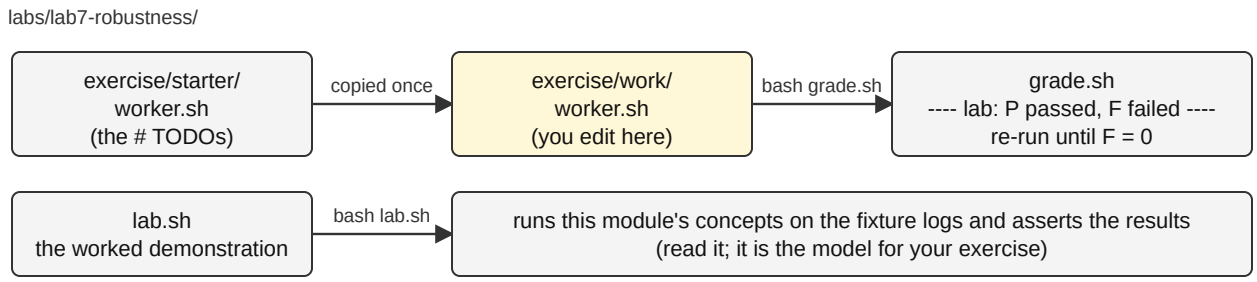
Lab 7 — Robustness: A Worker That Never Leaks and Never Lies About Its Exit Code (Instructor Version)

Instructor Note: Gray boxes and answers marked in red appear in the instructor copy only.

Topology



Environment Table

All paths are relative to `labs/lab7-robustness/exercise/`.

File	What it is	You edit it?
`TASK.md`	The short statement of the exercise	No
`starter/worker.sh`	Creates a temp dir and publishes its path; **no trap**; hard-coded `exit 0`	No — the grader copies it
`work/worker.sh`	**Your** copy (created by the first `grade.sh` run)	**Yes**
`grade.sh`	The grader: 6 checks. It runs your script several times and watches `/tmp`	No
`work/REFLECTION.md`	Your two reflection answers	**Yes** (you create it)

Objectives

- **Part 1:** Prepare the exercise folder and watch the starter leak.
- **Part 2:** Add a cleanup trap that preserves the exit code.
- **Part 3:** Make the exit code come from the caller.
- **Part 4:** Verify with the grader; write the two reflection answers.

Background / Scenario

A script that creates a temporary directory owes two promises: the directory is removed **however** the script ends — normal exit, `exit 3`, a `set -e` abort, Ctrl-C, `kill` — and the script's real exit code survives the cleanup. Both promises are kept by one `trap` line, and both are checked by `logsleuth`'s own tests. (The Course 1 script `scripts/replica-check.sh` shipped without that line until this course's audit found it; the fix is one statement.)

The contract for this lab: `worker.sh <marker> [exitcode]` creates a temp dir, writes its path to the file `<marker>` (so the grader can watch it), removes the dir on **any** exit or signal, and exits with `<exitcode>`

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

(default 0) — preserved **through** the cleanup trap.

Required Resources

- A lab session or a shell where `bash tools-check.sh` reports `ready`.
- Module M07, steps S07.2–S07.4.
- A text editor. Documentation and AI assistance for lookups are allowed (say so in your reflection). Do **not** open `solution/`.

> **Instructor Note:** Budget 50 minutes; grading ≈ 6 minutes per learner. Observed: the starter scores `---- lab7: 2 passed, 4 failed ----` — exit 0 happens to be "preserved" and the file is shellcheck-clean, but the temp dir leaks and exit 3 is ignored; the reference scores 6/6. Also glance at `ls /tmp/lab7.*` while grading: it should list nothing.

Part 1: Prepare the exercise folder and watch the leak

Step 1: Run the grader once.

```
cd labs/lab7-robustness/exercise
bash grade.sh
ls -d /tmp/lab7.* 2>/dev/null | wc -l
```

Expected: `---- lab7: 2 passed, 4 failed ----`, and a count of leaked `lab7.*` directories greater than zero — the starter creates a directory on every run and never removes it. Run the grader again and watch the count grow. Clean up before you continue:

```
rm -rf /tmp/lab7.*
```

Step 2: Open `work/worker.sh`.

The script already runs under `set -euo pipefail`, creates the directory with `mktemp -d`, and publishes its path. Two `# TODO` comments remain: the trap, and the exit code.

Part 2: Add the cleanup trap

Step 1: Install a trap that preserves the exit code.

Directly under the `# shellcheck disable=SC2154` comment (keep that comment — `rc` is assigned inside the trap string, and shellcheck cannot see it), add:

```
trap 'rc=$?; rm -rf "$d"; exit "$rc"' EXIT INT TERM
```

How it works, in order: `rc=$?` saves the exit code **first** — every command, including `rm`, replaces `$?`; `rm -rf "$d"` removes the directory; `exit "$rc"` ends the script with the code that was saved. `EXIT` fires on any exit, including a `set -e` abort; naming `INT` (Ctrl-C) and `TERM` (`kill`) turns those signals into the same cleanup path.

Note: Without the capture, a cleanup command that failed under `set -e` would end the script with status 1 — your `exit 3` would be lost. You observed exactly that in S07.3 (`with -e: status=1` versus `capture-and-reexit: status=3`).

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Part 3: Make the exit code come from the caller

Step 1: Replace the hard-coded exit.

Change the last line, `exit 0`, to:

```
exit "${2:-0}"
```

`$2` is the second argument; `${2:-0}` falls back to `0` when it is missing. Without the default, a call with one argument would die under `set -u` with `$2: unbound variable`.

Step 2: Try it by hand.

```
m=$(mktemp)
bash work/worker.sh "$m"; echo "status=$?"; test -d "$(cat "$m")" && echo LEAKED || echo "cleaned"
bash work/worker.sh "$m" 3; echo "status=$?"; test -d "$(cat "$m")" && echo LEAKED || echo "cleaned"
rm -f "$m"
```

Expected: `status=0` then `cleaned`; `status=3` then `cleaned`.

Part 4: Verify and reflect

Step 1: Run the grader until it is green.

```
bash grade.sh
```

Expected: `---- lab7: 6 passed, 0 failed ----`. The checks: exit 0 is preserved; temp dir removed after exit 0; exit 3 is preserved through the trap; temp dir removed after exit 3; three runs leave no `lab7.*` dirs behind; `worker.sh` is shellcheck-clean.

Symptom	Cause	Fix
`exit 3 is preserved through the trap` got `0`	The last line is still `exit 0`	`exit "${2:-0}"`
`temp dir removed after exit 0` fails	The trap line is still a comment	Add the `trap` line
shellcheck reports `rc is referenced but not assigned`	The `# shellcheck disable=SC2154` comment was removed	Restore it above the trap
`three runs leave no temp dirs behind` fails	Old leaks from before your fix	`rm -rf /tmp/lab7.*` and re-run

Step 2: Reflection.

Create `work/REFLECTION.md` and answer in your own words (3–5 sentences each). Both answers must include an observation from the module.

Question 1: Why does the trap capture `$?` into `rc` *before* `rm -rf`, and what would the grader's "exit 3 is preserved" check see if a cleanup command failed under `set -e`? Quote what you observed in S07.3.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

> **Answer:** **Met:** `$?` is overwritten by every command, so it must be saved first; under `set -e` a failing cleanup command inside the trap would abort the trap with status 1 and the caller would see `1` instead of `3`; quotes the observed `with -e: status=1` / `capture-and-reexit: status=3`. **Strong:** adds that `exit "$rc"` is what re-asserts the saved code, or that `logsleuth` guards twice (capture + a `cleanup` function ending in `return 0`). **Not met:** "to keep the exit code" with no mechanism or no observation.
>
> *Calibration.* Met: "The first thing in the trap is `rc=$?` because `$?` only holds the exit code until the next command runs. If `rm -rf` failed under `set -e`, the trap would stop and bash would exit 1 — in S07.3 I saw `status=1` for exactly that, and `status=3` once the code was captured and re-exited." Not met: "You save the return code so you can return it." — restates, no mechanism, no observation.

Question 2: The trap names `EXIT INT TERM`. Say what each name covers, and what you observed in S07.4 about a worker killed with `kill -TERM`.

> **Answer:** **Met:** `EXIT` = any normal or `set -e` exit; `INT` = Ctrl-C; `TERM` = `kill`'s default; naming the signals makes them run the same handler, which exits and so fires the cleanup; reports the observed `temp dir gone after SIGTERM`. **Strong:** notes one of the observed subtleties — the handler's status is the last command's (`0`), or that bash runs an EXIT-only trap on TERM anyway — and still justifies the course form. **Not met:** "to catch all signals" (SIGKILL cannot be trapped; HUP is not listed) or no observation.

Hand-in and grading

Hand in `work/worker.sh`, `work/REFLECTION.md` and your final grader output. **Pass** = `0 failed` **and** both reflection answers name the mechanism with an observation. Unlimited attempts.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Instructor Note: Grading procedure (≈6 min per learner).

1. `GRADE_TARGET=/path/to/their/work bash labs/lab7-robustness/exercise/grade.sh` → require `---- lab7: 6 passed, 0 failed ----`; glance at `ls /tmp/lab7.*` — nothing.
2. Read `REFLECTION.md` against the descriptors. Record `A-08 PASS` only with 6/6 and both answers at Met or better.
3. **Acceptable variations:** `trap 'rc=$?; rm -rf "$d"; exit "$rc"' EXIT` alone **passes** (the grader sends no signals) — accept it, but Question 2 must still explain `INT TERM`; the three-trap status-preserving form passes too (this grader does not count trap lines; the capstone's does — say so). `exit "$2"` without the `:-0` default **passes** (every grader call supplies a code) but breaks the contract's default: with one argument it dies under `set -u` with `$2: unbound variable`, status 1 (observed) — run `bash work/worker.sh "$(mktemp)"` by hand to catch it. Removing the `# shellcheck disable=SC2154` line makes shellcheck warn `rc is referenced but not assigned` and the quality check fails (observed 5/6).
4. **Model feedback (Not yet):** "`exit 3 is preserved through the trap` got `0`: your last line is still `exit 0`; it must be `exit "${2:-0}"`." / "`temp dir removed after exit 0` FAIL: the trap line is still a comment." Remediation: S07.2–S07.4.
5. **Equivalent reassessment:** same grader; Q1′ "Why does `mktemp -d` take a template ending in `XXXXXX`?" (Met: the X's are replaced by random characters so each run gets a unique, unpredictable name) and Q2′ "Why does the grader run the worker three times and count `/tmp/lab7.*` before and after?" (Met: a leak shows up as a growing count; three runs make a single accidental pass unlikely).

Instructor Note: Appendix — reference solution (`solution/worker.sh`, observed 6/6):

```
#!/usr/bin/env bash
# Lab 7 solution — signal-safe temp cleanup with a code-preserving trap.
# Contract:  worker.sh <marker> [exitcode]
set -euo pipefail

d="$(mktemp -d "${TMPDIR:-/tmp}/lab7.XXXXXX")"
# shellcheck disable=SC2154  # rc is assigned inside the trap string, then used
trap 'rc=$?; rm -rf "$d"; exit "$rc"' EXIT INT TERM   # INT/TERM funnel into EXIT

printf '%s\n' "$d" > "$1"     # publish our temp dir so the grader can watch it

exit "${2:-0}"
```

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.